TEHNOLOGII WEB

Fișa de laborator nr. 4

Date de la server și formulare cu validare

Proiect: City Explorer

Nume	Prenume	Grupa	Data
Caciularu	Stefan-Arthur	633AB	10.11.2026

Cum se completează

Fișa se predă împreună cu aplicația care funcționează. Punctele de mai jos corespund punctelor marcate `Fișă` din lucrare și se completează **în timpul laboratorului**, pe măsură ce se măsoară: o valoare citită în fila Network și nescrisă atunci se pierde.

Ce este predicția. Fiecare rând al fișei începe cu o predicție: răspunsul scris **înainte** de a rula, împreună cu motivul pentru care îl aștepți. Nu este o ghicire și nu se notează dacă este corectă sau greșită. Rolul ei este să te facă să te uiți la cod înainte de a te uita la rezultat, pentru că lucrul care se învață este **diferența** dintre ce ai crezut că se întâmplă și ce se întâmplă în realitate. O predicție greșită, urmată de explicația corectă a diferenței, valorează mai mult decât o fișă în care totul s-a potrivit din prima.

Fiecare măsurătoare se face în trei pași, în ordinea aceasta:

1. **se scrie întâi predicția**, înainte de a rula: ce se așteaptă să se întâmple și de ce;
2. **apoi se rulează și se notează rezultatul obținut**, cu valoarea măsurată;
3. dacă cele două diferă, se scrie în două rânduri de ce.

Un rând completat corect arată așa:

Măsurătoarea	Predicție	Valoare măsurată	Dacă diferă, de ce
Exemplu: cereri la o singură reîncărcare	O singură cerere. `useEffect` are lista de dependențe goală, deci ar trebui să ruleze o dată.	2	În dezvoltare, `<StrictMode>` montează componenta de două ori, intenționat. La `npm run build` pleacă o singură cerere.

Atenție: Predicția scrisă după măsurătoare se recunoaște imediat, pentru că este întotdeauna corectă. Coloana a patra este cea care se notează, nu a treia.

Pregătirea instrumentelor

Se face o singură dată, la începutul laboratorului. Toate măsurătorile se citesc apoi din aceleași două file.

1. Două terminale deschise în folderul proiectului: în primul `npx json-server db.json --port 3001`, în al doilea `npm run dev`. Niciunul nu se închide până la final.
2. În browser, pe pagina aplicației, se apasă `F12`. Se deschide fila **Network**.

3. Sub bara filei, se apasă butonul `Fetch/XHR`. Fără el, în listă apar și fișierele paginii, câteva zeci, iar cererile nu se mai pot număra.
4. Se bifează **Disable cache**, în aceeași bară. Altfel, de la a doua reîncărcare, răspunsul vine din memoria browserului, iar coloanele Size și Time arată alte valori.
5. Dacă în capul tabelului nu apar coloanele **Status**, **Size** și **Time**, se dă click dreapta pe el și se bifează.
6. Butonul de golire a listei este cercul tăiat, în stânga barei. Se apasă înainte de fiecare măsurătoare.
7. Numărul total de cereri din listă se citește în bara de jos a filei, nu se numără cu ochiul.
8. Fila **Console**, de lângă Network, se folosește la punctele 4 și 5. Comanda se scrie acolo și se apasă `Enter`.

Atenție: Fiecare măsurătoare începe cu golirea listei. O listă nemăsurată, în care au rămas cererile de la pasul anterior, dă un număr mai mare și o fișă greșită la toate rândurile care urmează.

1. Numărul de cereri trimise către server

Exercițiul 1, punctul d. Cu Network deschis și filtrul pe `Fetch/XHR`, se reîncarcă pagina.

Rândul din fișă	Pașii, în ordine	Unde se citește și ce formă are rezultatul
1. Cereri la o reîncărcare	1. Se golește lista Network. 2. Se apasă `F5`. 3. Se numără rândurile cu numele `destinatii`.	În listă. Un număr întreg mic. Dacă nu este cel prezis, nu este o eroare de configurare: se deschide `src/main.jsx` și se caută acolo explicația, apoi se scrie în ultima coloană a fișei.
2. Durata cererii	1. Se dă click pe rândul cererii. 2. Se citește coloana **Time**. 3. Se reîncarcă pagina de trei ori și se notează cele trei valori.	Coloana **Time**. O valoare în milisecunde, de ordinul unităților sau al zecilor. Media se calculează de mână, din cele trei.
3. Cereri în cinci secunde, fără `[]`	1. În `src/App.jsx` se șterge `, []`, al doilea argument al lui `useEffect`, și se salvează. 2. Se golește lista. 3. Se numără cinci secunde și se pune `[]` la loc, imediat.	În bara de jos a filei Network, câmpul cu numărul total de cereri. Un număr mare, care continuă să crească atâta timp cât `[]` lipsește.
4. Cereri în timp ce se tastează	1. Se golește lista. 2. Se scrie `portugalia` în câmpul de căutare, literă cu literă. 3. Se privește lista Network în timp ce se tastează.	În listă. Se notează câte rânduri noi au apărut. Se scrie în fișă și motivul, adică unde se face filtrarea de la exercițiul 3.

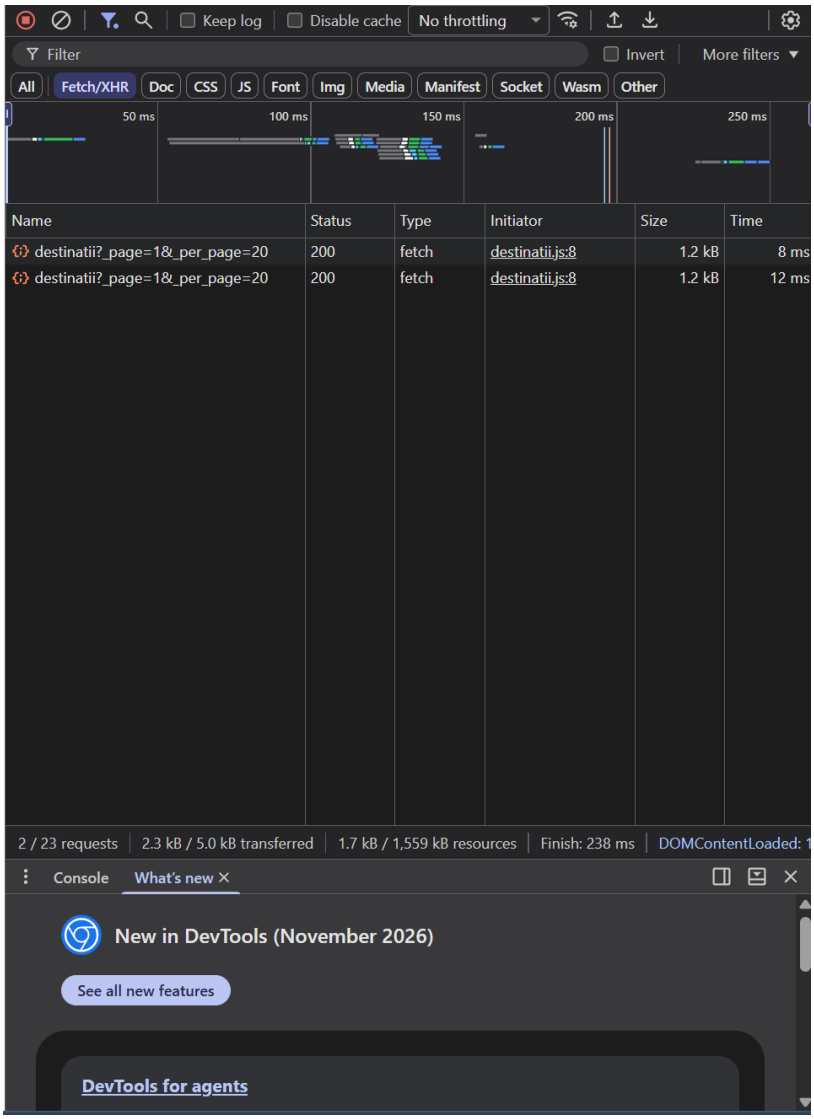
Atenție: Măsurătoarea a treia pornește sute de cereri pe secundă. Se ține cinci secunde, nu mai mult, iar `[]` se pune la loc imediat. Numărul diferă de la calculator la calculator; nu există o valoare „corectă” care să poată fi copiată de la coleg.

Măsurătoarea	Predicție	Valoare măsurată	Dacă diferă, de ce
1. Cereri la o singură reîncărcare	2 cereri GET /destinatii. Efectul are o listă de dependențe care nu se schimbă la pornire, deci ar rula o dată, dar în `src/main.jsx` aplicația este învelită în `<StrictMode>`, care în dezvoltare montează componenta de două ori.	2 (captura de mai jos: două rânduri `destinatii?_page=1&_per_page=20`, status 200)	Nu diferă. Dublarea vine din `<StrictMode>` și apare doar în modul de dezvoltare; la `npm run build` pleacă o singură cerere.

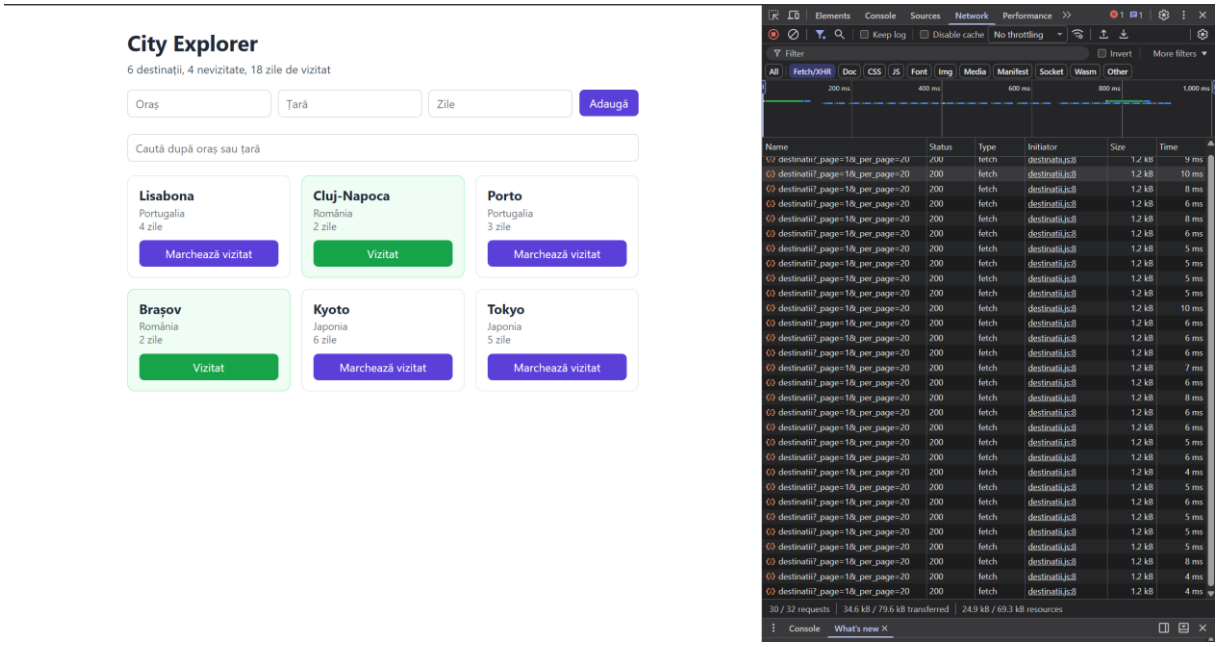
Măsurătoarea	Predicție	Valoare măsurată	Dacă diferă, de ce
2. Durata cererii: cele trei valori și media	Câteva milisecunde, sub 20 ms: serverul rulează pe același calculator și răspunsul are sub 1 kB, deci nu există o rețea reală de traversat.	8 ms, 8 ms, 5 ms media: 7 ms	Nu diferă: media de 7 ms este sub 20 ms. Serverul e pe același calculator, deci timpul înseamnă doar pregătirea și citirea unui răspuns de 1,2 kB.
3. Cereri în cinci secunde, fără `[]`	Sute de cereri, fără oprire. Fără al doilea argument, efectul rulează după fiecare desenare; fiecare răspuns apelează `setDestinatii` cu o listă nouă, care declanșează o desenare nouă, care rulează din nou efectul: o buclă infinită.	30 de cereri (bara de jos: „30 / 32 requests”), toate `destinatii?_page=1&_per_page=20`, cu status 200.	Am măsurat mai puține decât am prezis: în captură, cererile se întind pe aproximativ o secundă (axa de timp ajunge la 1000 ms), deci am pus `[pagina]` la loc mai repede de 5 secunde. Ritmul de aproximativ 30 de cereri pe secundă confirmă bucla: o cerere nouă pleacă imediat după fiecare răspuns.
4. Cereri în timp ce se tastează în căutare	0 cereri. Filtrarea de la exercițiul 3 se face în `App`, cu `filter` pe lista deja primită, nu pe server.	0	Nu diferă. Textul căutat este o stare: la fiecare tastă `App` se redesenează, dar efectul depinde doar de `[pagina]`, care nu se schimbă, deci nu pleacă nicio cerere.

Valorile de mai sus se dovedesc cu două capturi, făcute cu `Win + Shift + S` și lipite în celulele de mai jos. Fără ele, măsurătorile 1, 2 și 3 nu se iau în considerare.

Captură de ecran. Fila Network după o reîncărcare, cu filtrul pe `Fetch/XHR`: trebuie să se vadă rândurile cererilor și coloana Time.



Captură de ecran. Fila Network după cele cinci secunde fără `[]`: trebuie să se vadă numărul total de cereri, din bara de jos a filei.



2. Conținutul cererii și al răspunsului

Exercițiul 2, punctul d. Se adaugă o destinație corectă și se deschide cererea `POST` din Network: fila **Payload** arată corpul trimis, fila **Response** ce a răspuns serverul.

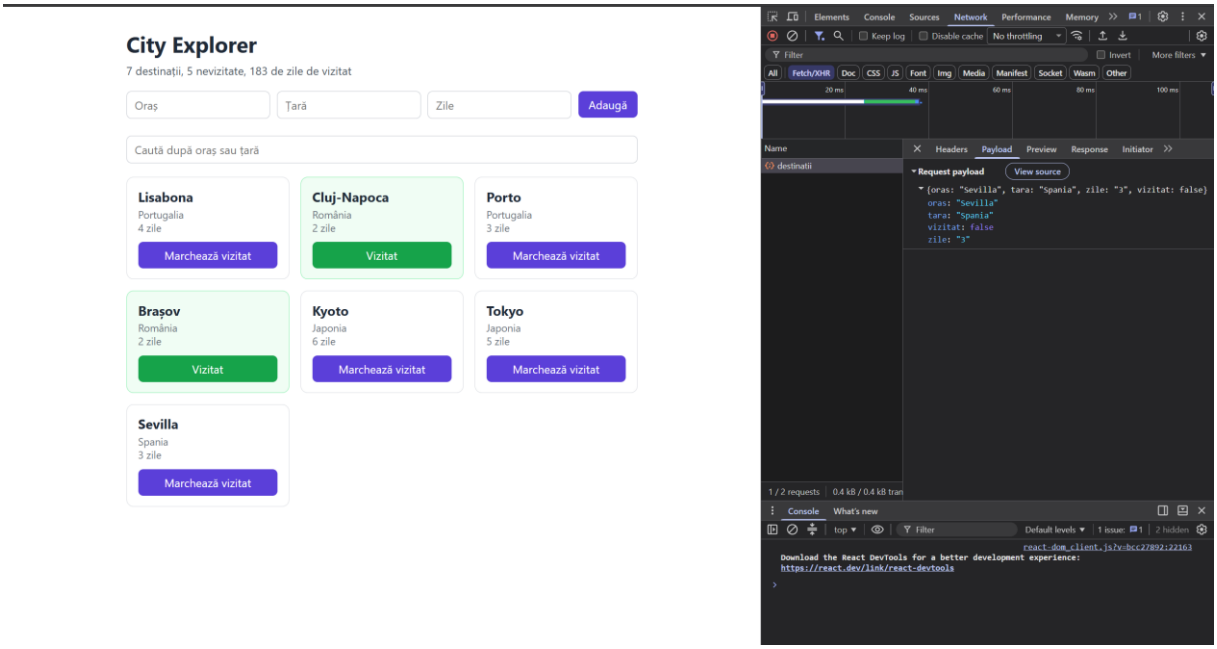
Rândul din fișă	Pașii, în ordine	Unde se citește și ce formă are rezultatul
1. Câmpurile din Payload și cele din Response	1. Se golește lista și se adaugă o destinație corectă din formular. 2. Se dă click pe rândul `POST` apărut în listă. 3. Se deschide fila **Payload** și se notează numele câmpurilor. 4. Se trece pe fila **Response** și se notează și acolo.	În panoul din dreapta cererii. Două liste de nume de câmpuri. Se scrie în fișă și care este diferența dintre ele, adică ce câmp apare într-una și nu în cealaltă.
2. Tipul lui `zile`, cu `Number(...)` și fără	1. În componenta formularului se găsește linia `zile: Number(valori.zile),` și se schimbă în `zile: valori.zile,`. 2. Se adaugă o destinație cu 3 zile. 3. Se citește Payload-ul cererii `POST`. 4. Se privește rezumatul din capul paginii. 5. Se pune `Number(...)` la loc și se șterge destinația adăugată din `db.json`.	În fila **Payload**, pe linia `zile`. Se notează dacă valoarea apare cu sau fără ghilimele, în ambele variante. În rezumat, se notează numărul afișat după adăugare. Nu apare nicio eroare în consolă; dacă apare, s-a greșit altceva.
3. Statusul pentru `GET`, `POST` și `PATCH`	1. Fără a goli lista după adăugare, se apasă butonul de vizitat pe un card, ca să plece și un `PATCH`. 2. Se citește coloana **Status** pe fiecare dintre cele trei rânduri.	Coloana **Status**. Trei numere de trei cifre. Se scrie și ce înseamnă fiecare; dacă două sunt identice și unul diferit, diferența este chiar răspunsul cerut.

Observație: A doua măsurătoare arată de ce există `Number(...)`: rezumatul nu semnalează nicio eroare, doar afișează un număr aproape corect. În JavaScript, greșelile de tip se manifestă rareori ca erori, ci ca rezultate greșite.

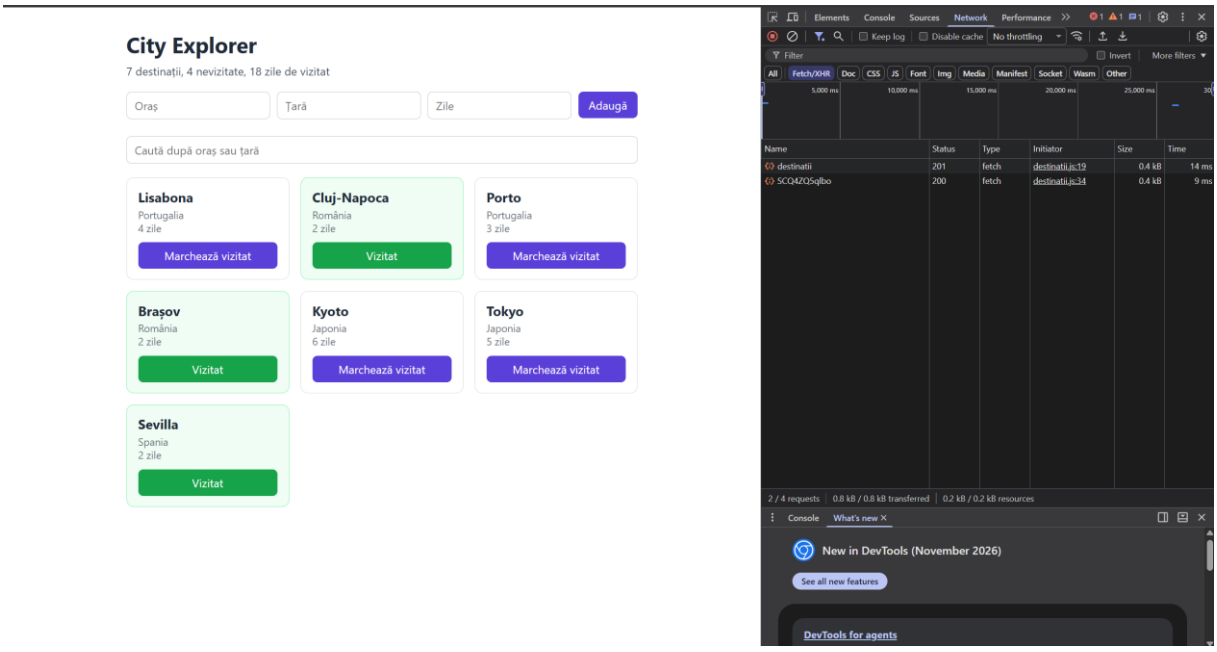
Măsurătoarea	Predicție	Valoare măsurată	Dacă diferă, de ce
1. Câmpurile din Payload și cele din Response	Payload: `oras`, `tara`, `zile`, `vizitat`. Response: aceleași câmpuri plus `id`, pentru că identificatorul îl generează serverul.	Payload: `oras`, `tara`, `zile`, `vizitat`. Response: `id`, `oras`, `tara`, `zile`, `vizitat`; `id` este un șir generat de server, la mine `SCQ4ZQ5qlbo`.	Nu diferă. Diferența dintre ele este câmpul `id`. De aceea în stare se pune obiectul din Response: fără `id`, cardul nou nu ar avea `key` și nu s-ar putea trimite `PATCH /destinatii/:id` pentru el.

Măsurătoarea	Predicție	Valoare măsurată	Dacă diferă, de ce
2. Tipul lui `zile`, cu `Number(...)` și fără	Cu `Number(...)`: `3`, număr, fără ghilimele. Fără: `"3"`, text. Rezumatul ar trebui să afișeze un total greșit.	Cu `Number(...)`: `"zile": 2` (Sevilla, 2 zile); rezumatul: „7 destinații, 5 nevizitate, 20 de zile de vizitat”. Fără: `zile: "3"`, cu ghilimele; rezumatul: „7 destinații, 5 nevizitate, 183 de zile de vizitat”. După încă o adăugare: „8 destinații, 5 nevizitate, 1533 de zile de vizitat”.	Tipul l-am prezis corect, dar nu și efectul. `reduce` adună numere până la Sevilla (0 + 4 + 3 + 6 + 5 = 18), apoi `18 + "3"` lipește textele: „183”. La a doua Sevilla lipește din nou, „1533”. Nu apare nicio eroare în consolă, doar un număr care pare plauzibil.
3. Statusul pentru `GET`, `POST` și `PATCH`	200 pentru toate trei: toate cererile reușesc.	GET: 200 POST: 201 PATCH: 200 (cererea cu id-ul `SCQ4ZQ5qlbo`)	POST întoarce 201 Created, nu 200: serverul semnalează că a creat o resursă nouă. GET și PATCH întorc 200 OK, adică cererea a reușit pe o resursă existentă.

Captură de ecran. Fila Payload a cererii `POST`, în varianta fără `Number(...)`: trebuie să se vadă `"zile": "3"`, cu ghilimele.



Captură de ecran. Coloana Status din Network, cu cele trei cereri, `GET`, `POST` și `PATCH`, vizibile în aceeași listă.

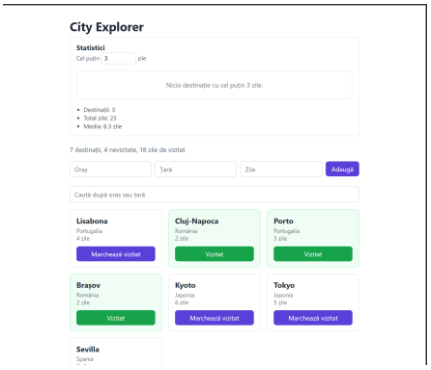


3. Cele șapte greșeli din panoul de statistici

Exercițiul 4. Componenta primită conține șapte greșeli, cinci în cod și două în stil. Se pornește de la ce se vede pe ecran și se merge înapoi spre cauză. Pentru fiecare simptom se completează un rând. Captura se face cu `Win + Shift + S` și se lipește în a doua celulă, sub textul scris; rândurile se pot mări.

Cele șapte nu se găsesc citind codul de la cap la coadă. Fiecare are un instrument propriu, iar ordinea de mai jos este cea în care se găsesc cel mai repede:

1. **Se încearcă interfața.** Se scrie o altă valoare în câmpul pragului și se privește ce se întâmplă cu panoul. Un simptom se vede aici.
2. **Se oprește serverul** cu `Ctrl+C` în terminalul lui `json-server` și se reîncarcă pagina. Alt simptom se vede aici. Apoi se repornește.
3. **Se citește ecranul ca un utilizator.** Două elemente care nu ar trebui să apară în același timp apar împreună; iar caseta arată altfel decât în figura 1.
4. **Se verifică numerele** față de `db.json`. Se deschide fișierul alături și se adună de mână zilele destinațiilor care ar trebui să intre în socoteală, la pragul afișat. Două simptome se găsesc numai așa, pentru că nu lasă nicio urmă în consolă.
5. **Se deschide fila Elements.** Click dreapta pe caseta panoului, apoi `Inspect`. În panoul **Styles** din dreapta se caută declarațiile **tăiate cu o linie**: o declarație tăiată este una scrisă de cineva și neaplicată de browser. Ultimele două simptome se explică de aici.

Simptomul	Ce am observat, cu captura dedesubt	Cum afectează aplicația	Cauza: fișier, linie, corectură
1. Numerele nu se potrivesc între ele: destinațiile, totalul de zile și media.	La pragul 3 panoul arată „Destinații: 3, Total zile: 25, Media: 8.3 zile”. 25 este suma zilelor tuturor celor 7 destinații, dar cele 3 numărate (Lisabona 4, Kyoto 6, Tokyo 5) au doar 15 zile. 	Utilizatorul vede o medie de 8,3 zile în loc de 4,2 și ar planifica vacanțe de două ori mai lungi. Numărul pare plauzibil, deci nimic nu-l avertizează.	`src/components/PanouStatistici.jsx`, linia 38: totalul se calcula pe toate destinațiile. Corectură: `filtrate.reduce((suma, d) => suma + d.zile, 0)`. Corect, la pragul 3: 5 destinații, 21 de zile, media 4.2.

Simptomul	Ce am observat, cu captura dedesubt	Cum afectează aplicația	Cauza: fișier, linie, corectură
2. Mesajul „nicio destinație” apare în același timp cu lista de statistici.	În aceeași captură: „Nicio destinație cu cel puțin 3 zile.” apare chiar deasupra listei „Destinații: 3”.	Panoul se contrazice singur: utilizatorul nu știe dacă există destinații sau nu. Când lista chiar e goală, în locul mesajului apare cifra 0.	Același fișier, linia 55: `{filtrate.length && (...)}`; 3 este adevărat, deci se afișează mesajul pentru lista goală. Corectură: `{filtrate.length === 0 && (...)}`.
3. La pragul 3, o destinație din `db.json` lipsește din socoteală.	În `db.json`, cu cel puțin 3 zile sunt Lisabona (4), Porto (3), Kyoto (6), Tokyo (5) și Sevilla (3): 5 destinații. Panoul numără 3; lipsesc Porto și Sevilla, care au exact 3 zile.	Orice destinație aflată exact la prag dispare din statistici. Textul promite „cel puțin”, dar calculul face „mai mult de”.	Linia 37: `d.zile > Number(prag)`. Corectură: `d.zile >= Number(prag)`.
4. La scrierea altei valori în câmpul pragului, panoul nu mai răspunde.	Fără captură. Din cod: `onChange={setPrag}` pune în stare evenimentul în loc de text, deci după prima tastă câmpul se golește, statisticile dispar și apare cifra 0. Pentru captura cu pragul 5 am schimbat valoarea inițială direct în cod: `useState('5')` dă „Destinații: 1, Total zile: 25, Media: 25.0 zile”.	Singura funcție a panoului, schimbarea pragului, nu mai merge deloc; utilizatorul rămâne cu un panou blocat, fără niciun mesaj de eroare.	Linia 50: `onChange={setPrag}` pune în stare obiectul eveniment, nu textul; `Number(eveniment)` dă `NaN`, deci nicio destinație nu trece filtrul. Corectură: `onChange={(event) => setPrag(event.target.value)}`.
5. Cu `json-server` oprit, panoul rămâne pe mesajul de încărcare.	Fără captură: cu serverul oprit, `App` afișează doar eroarea („Nu am putut încărca datele: Failed to fetch”), deci panoul nu ajunge pe ecran. Din cod: `setIncarcare(false)` lipsește din ramura de eroare, deci panoul ar rămâne pe „Se încarcă statisticile...”.	Utilizatorul așteaptă degeaba și nu află că serverul este oprit, deci nu are de ce să reîncerce sau să anunțe problema.	Linia 20: `setIncarcare(false)` era doar în `try`, deci la eroare nu se executa, iar `if (incarcare)` se verifică înaintea lui `if (eroare)`. Corectură: `setIncarcare(false)` mutat într-un bloc `finally`, după `catch`.

Simptomul	Ce am observat, cu captura dedesubt	Cum afectează aplicația	Cauza: fișier, linie, corectură
6. Colțurile casetei sunt drepte, nu rotunjite ca în figura 1.	Fără captură în Styles. Pe ecran, caseta panoului are colțuri drepte; în cod, `border-radius: 10;` nu are unitate, deci browserul o ignoră.	Doar aspectul: panoul nu seamănă cu cardurile din restul aplicației. Datele nu sunt afectate.	`src/App.css`, regula `.panou`: `border-radius: 10;` fără unitate este o valoare invalidă și browserul o ignoră. Corectură: `border-radius: 10px;`.
7. Titlul **Statistici** este lipit de câmpul de sub el.	Fără captură în Styles. Pe ecran, titlul „Statistici” e lipit de câmpul de dedesubt; în cod, a doua regulă `.panou h2 { margin: 0; }` anulează `margin-bottom: 0.6rem`.	Doar aspectul: panoul arată înghesuit față de figura 1.	`src/App.css`, a doua regulă `.panou h2 { margin: 0; }`, de la finalul blocului: are aceeași specificitate și este scrisă mai jos, deci câștigă și anulează `margin-bottom`. Corectură: am șters a doua regulă.

Așa trebuie să arate panoul după cele șapte corecturi, cu pragul inițial:

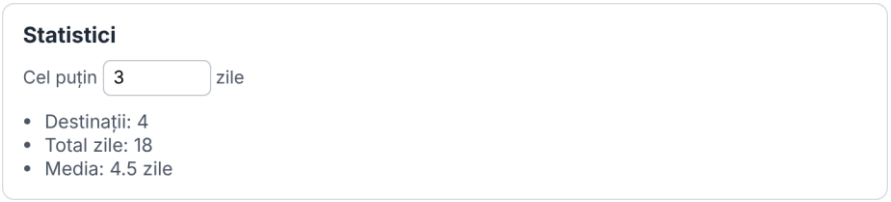


Figura 1. *Panoul de statistici, în forma corectă. Cele patru destinații cu cel puțin 3 zile însumează 18 zile, cu o medie de 4,5.*

Captură de ecran. Fila Elements, cu caseta panoului selectată și panoul Styles deschis: trebuie să se vadă declarația care nu se aplică, tăiată cu o linie. Una pentru greșeala 6 sau 7, la alegere.

Atenție: Coloana a treia nu se completează cu „nu merge”. Se scrie ce vede greșit utilizatorul și ce decizie ar lua pe baza unui număr greșit. O medie falsă este mai gravă decât o pagină oprită, pentru că pagina oprită se observă.

4. Măsurători pe varianta nepaginată

Exercițiul 5, punctul b. Cu cele două mii de destinații încărcate de pe portul `3002`, cu Network deschis și filtrul pe `Fetch/XHR`, se reîncarcă pagina.

Rândul din fișă	Pașii, în ordine	Unde se citește și ce formă are rezultatul
1. Dimensiunea răspunsului	1. Se verifică că **Disable cache** este bifat. 2. Se golește lista și se reîncarcă pagina. 3. Se dă click pe rândul `destinatii`.	Coloana **Size**. Se notează în kiloocteți. Dacă apare `(disk cache)` sau o valoare de câteva kiloocteți, răspunsul a venit din memoria browserului: se bifează **Disable cache** și se reîncarcă.
2. Durata cererii	Se citește pe același rând, fără altă acțiune.	Coloana **Time**, în milisecunde.
3. Carduri în pagină	1. Se trece pe fila **Console**. 2. Se scrie comanda de mai jos și se apasă `Enter`.	Numărul întors de consolă, pe linia de sub comandă. Este numărul de elemente existente în pagină, nu cel de carduri vizibile pe ecran.
4. Prima tastare în căutare	1. În consolă se rulează prima linie de mai jos. 2. Se trece imediat în câmpul de căutare și se scrie o literă. 3. Se revine în consolă și se rulează a treia linie.	Valoarea întoarsă de a treia linie, în milisecunde. Conține și timpul de mutare a mâinii, deci se repetă de două sau trei ori și se ia cea mai mică valoare.

Punctul 3, se scrie în fila Console

```
document.querySelectorAll('.card').length
```

Punctul 4, se scrie în fila Console

```
const t0 = performance.now()
// se tastează o literă în câmpul de căutare, apoi, imediat:
performance.now() - t0
```

Observație: Mai exact decât cronometrarea manuală: fila **Performance**, înregistrare, o tastare, oprire. Durata blocului galben, „Scripting”, este timpul cerut.

Măsurătoarea	Predicție	Valoare măsurată	Dacă diferă, de ce
1. Dimensiunea răspunsului, în kiloocteți	În jur de 200 kB: 2000 de obiecte de aproximativ 100 de caractere fiecare.	nemăsurat; în lucrare (figura 8): 224 kB	Nu am măsurat setul de 2000 de destinații. Valoarea din lucrare este apropiată de predicție: mărimea vine din cele 2000 de obiecte, trimise o singură dată.
2. Durata cererii, în milisecunde	Zeci de milisecunde, mai mult decât la cele 6 destinații, pentru că serverul are de citit și de trimis aproximativ 200 kB.	nemăsurat	nemăsurat

Măsurătoarea	Predicție	Valoare măsurată	Dacă diferă, de ce
3. Carduri în pagină	2000: fără paginare, fiecare destinație primită devine un card.	nemăsurat; după cod: 2000	Nu am măsurat. Fără paginare, fiecare destinație primită devine un card, deci în pagină ar fi 2000 de elemente `.card`, chiar dacă pe ecran se văd doar câteva.
4. Prima tastare în căutare, în milisecunde	Zeci până la sute de milisecunde: la fiecare tastă `App` se redesenează și refiltrează toate cele 2000 de carduri.	nemăsurat	nemăsurat

Captură de ecran. Rândul cererii `GET /destinatii` în Network, pe setul de două mii, cu coloanele Size și Time vizibile.

Captură de ecran. Consola, cu rezultatul lui `document.querySelectorAll('.card').length` și cu durata primei tastări.

5. Măsurători pe varianta paginată

Exercițiul 5, punctul d. Se repetă cele patru măsurători, cu aplicația paginată. Coloana „Nepaginat” se copiază din punctul 4 al fișei.

1. Se verifică întâi că paginarea funcționează: butoanele **Înapoi** și **Înainte** schimbă cardurile, iar textul dintre ele arată pagina curentă și numărul total de pagini.
2. Se repetă, în aceeași ordine, cei patru pași de la punctul 4, în aceeași sesiune de browser și pe același calculator. Măsurătorile de pe calculatoare diferite nu se pot compara.
3. Se completează coloana **Paginat**, apoi se calculează ultima coloană: valoarea nepaginată împărțită la cea paginată, rotunjită.
4. La final se pune portul înapoi pe `3001` în `src/api/destinatii.js`, ca aplicația să revină la cele șase destinații.

Ce se măsoară	Predicție	Nepaginat	Paginat	De câte ori mai mic
Dimensiunea răspunsului	În jur de 2,5 kB, de aproximativ 100 de ori mai mic: se trimit 20 de destinații din 2000.	224 kB (din lucrare)	≈ 2,5 kB (din lucrare)	≈ 90
Durata cererii	Câțiva ms, apropiat de cererea cu 6 destinații.	nemăsurat	nemăsurat	–
Carduri în pagină	20, câte cere `_per_page=20`.	2000 (după cod)	20 (după cod)	100
Prima tastare în căutare	Câțiva ms: se redesenează doar 20 de carduri.	nemăsurat	nemăsurat	–

Ultima coloană se calculează din propriile valori: valoarea nepaginată împărțită la cea paginată.

Captură de ecran. Același rând din Network, în varianta paginată, cu Size și Time vizibile.

Captură de ecran. Consola, în varianta paginată: numărul de carduri și durata primei tastări.

6. Concluzia

Exercițiul 5, punctul e. Cele trei întrebări nu se măsoară, se argumentează cu valorile deja scrise în fișă. Unde se caută răspunsul pentru fiecare:

1. Pentru prima, se compară rândul **prima tastare** de la punctele 4 și 5: pragul se deduce de la mărimea listei la care întârzierea a început să se simtă, nu din dimensiunea răspunsului.
2. Pentru a doua, cu paginarea pornită, se privește rezumatul din capul paginii și se caută în câmpul de căutare un oraș aflat pe pagina a cincea. Ce se vede acolo este răspunsul.
3. Pentru a treia, se compară valorile de la punctul 5 cu cele ale aplicației cu șase destinații, de la punctele 1 și 2 ale fișei.

Întrebare	Răspuns
Începând de la câte destinații se justifică paginarea? Răspunsul se argumentează cu valorile măsurate, nu cu o impresie.	De la câteva sute de destinații. Timpii pe setul de 2000 nu i-am măsurat, deci argumentez din ce am măsurat și din numărul de carduri: cu 6 destinații cererea durează 7 ms și se desenează 6 carduri; fără paginare s-ar desena 2000 la fiecare tastă, de peste 300 de ori mai multe, iar răspunsul ar crește de la 1,2 kB la aproximativ 224 kB. Costul care se simte este desenarea la fiecare tastă, care crește cu numărul de carduri, nu descărcarea, care se face o singură dată.
Ce funcționalitate se pierde odată cu paginarea? Se numește concret comportamentul care nu mai este disponibil.	Căutarea și rezumatul lucrează doar pe pagina curentă. Pe pagina 1, căutarea unui oraș aflat pe pagina 5, de exemplu „Lisabona 100”, afișează „Niciun rezultat”, deși orașul există pe server. Rezumatul arată doar cele 20 de destinații ale paginii, nu cele 2000.
Pentru cele șase destinații ale aplicației, paginarea ar aduce vreun câștig? Se motivează răspunsul.	Nu. Cu 6 destinații răspunsul are 1,2 kB și cererea durează în medie 7 ms (punctul 1), iar 6 carduri se desenează instantaneu: paginarea nu ar avea ce reduce. În schimb, ar adăuga butoane inutile și ar limita căutarea și rezumatul la o pagină, adică pierderi fără niciun câștig.